



---

# FİNAL TASARIM RAPORU

---

Ders: BİL 204 / YAZILIM MÜHENDİSLİĞİ  
Bölüm/Şube: BİLGİSAYAR MÜHENDİSLİĞİ / 2.SINIF  
Grup Adı: Code Sentinel  
Proje Adı: Statik Kod Analizi ve Hata Tespit Sistemi  
Teslim Tarihi: 24.05.2026

## Grup Üyeleri:

- Ferhat Enes KOCABIYIK (24100011813)
- Hacı Ömer OĞUZ (24100011082)
- Hüseyin Mert KANLIDERE (24100011021)
- Kemal BABAOĞLAN (25100011472)
- Mehmet Enes İYİŞENYÜREK (23100011005)
- Mehmet UÇAR (24100011044)

## İçindekiler

|                                            |    |
|--------------------------------------------|----|
| 1. Giriş.....                              | 3  |
| Projenin Temel Özellikleri.....            | 3  |
| 2. Proje Deneyimi .....                    | 3  |
| Çizilen UML Diyagramları.....              | 4  |
| 3. Yapay Zeka Kullanımı.....               | 5  |
| 3.1. Kullanılan Yapay Zeka Modelleri ..... | 5  |
| 3.2. Kullanım Alanları .....               | 5  |
| 3.3. Yapay Zekanın Katkısı .....           | 5  |
| 4. Kullanıcı Kılavuzu .....                | 6  |
| Temel Kullanım Akışı.....                  | 6  |
| 1.Giriş Yapma .....                        | 7  |
| 2. Ana Arayüz .....                        | 7  |
| 3. Dosya Seçme İşlemi.....                 | 8  |
| 4. Analiz Sonuçları.....                   | 9  |
| 5. Filtreleme Sistemi .....                | 10 |
| 6.Rapor Dışa Aktarma .....                 | 11 |
| Kullanım Adımları .....                    | 11 |
| 5. Kurulum Talimatları .....               | 13 |
| 6. Görev Dağılımı .....                    | 14 |
| 7. Yapılanlar ve Yapılamayanlar .....      | 14 |
| Alt Sistemler .....                        | 14 |
| Analiz Süreci.....                         | 14 |
| Tamamlanan Özellikler.....                 | 15 |
| Tamamlanamayan Özellikler .....            | 15 |
| 8. Sonuç ve Genel Değerlendirme .....      | 15 |
| 9. GitHub Repository .....                 | 16 |

## 1. Giriş

Bu proje kapsamında C programlama dili için çalışan masaüstü tabanlı bir statik kod analiz sistemi geliştirilmiştir. Sistem, kullanıcı tarafından yüklenen .c uzantılı kaynak kod dosyalarını çalıştırmadan analiz ederek syntax hatalarını, kod kalitesi problemlerini ve bazı güvenlik risklerini tespit etmektedir.

Projenin temel amacı, yazılım geliştirme sürecinde oluşabilecek hataları daha erken aşamada tespit ederek geliştiricilere hızlı geri bildirim sağlamaktır. Özellikle C dilinin düşük seviyeli yapısı nedeniyle oluşabilecek bellek yönetimi problemleri ve syntax hatalarının çalışma zamanına ulaşmadan önce belirlenmesi hedeflenmiştir.

Sistem; dosya doğrulama, yorum satırlarını ayıklama, tokenization, parser, AST oluşturma, syntax analyzer, rule engine, metric calculator ve report manager gibi modüllerden oluşmaktadır. Analiz sonuçları kullanıcı arayüzünde görüntülenebilmekte ve TXT, JSON veya HTML formatlarında dışa aktarılabilir.

### Projenin Temel Özellikleri

- C kaynak kodlarını analiz etme
- Syntax hatalarını tespit etme
- Kural bazlı statik analiz gerçekleştirme
- LOC, fonksiyon sayısı ve struct sayısı gibi metrikleri hesaplama
- TXT, JSON ve HTML raporları oluşturma
- Critical, warning ve info seviyesinde hata sınıflandırması yapma

## 2. Proje Deneyimi

Proje geliştirme süreci analiz, tasarım ve implementasyon olmak üzere üç temel aşamada yürütülmüştür. İlk aşamada sistem gereksinimleri detaylı şekilde belirlenmiş ve functional/non-functional gereksinimler hazırlanmıştır.

Analiz raporu hazırlanırken kullanıcı ile sistem arasındaki etkileşimler Use Case diyagramları üzerinden modellenmiştir. Sistemin yükleme, analiz başlatma, syntax kontrolü, rapor oluşturma ve sonuç görüntüleme gibi süreçleri ayrı kullanım senaryoları halinde detaylandırılmıştır.

Tasarım aşamasında sistemin modüler bir mimariye sahip olması hedeflenmiştir. Bu nedenle katmanlı mimari yaklaşımı benimsenmiş ve sistem bağımsız alt modüllere ayrılmıştır.

Tasarım sürecinde Use Case, Sınıf Diyagramı, Sequence Diyagramı, Aktivite Diyagramı, Durum Diyagramı hazırlanmıştır. Bu diyagramlar sayesinde sistemin hem mantıksal hem de fiziksel yapısı detaylı şekilde modellenmiştir.

Implementasyon aşamasında GitHub üzerinden branch mantığı kullanılmıştır. Her ekip üyesi kendi modülleri üzerinde bağımsız şekilde çalışmış ve değişiklikler Pull Request yöntemiyle projeye dahil edilmiştir.

Bu proje sayesinde ekip üyeleri yalnızca kod geliştirme değil, aynı zamanda yazılım mühendisliği süreçleri, ekip çalışması, sürüm kontrol sistemleri ve proje planlama konusunda da önemli deneyimler kazanmıştır.

### **Çizilen UML Diyagramları**

- Use Case Diyagramı
- Sınıf Diyagramı
- Sequence Diyagramı
- Aktivite Diyagramı
- Durum Diyagramı

### 3. Yapay Zeka Kullanımı

#### 3.1. Kullanılan Yapay Zeka Modelleri

- Open AI ChatGPT
- Antropic Claude
- Google Gemini

#### 3.2. Kullanım Alanları

- Yazdığımız Word / PDF belgelerinin biçiminin , yazım kurallarının ve cümle yapısının düzeltilmesi.
- HTML ve JSON çıktılarını oluşturan kodların yazılması.
- GUI tasarımı ve kodu.
- Uygulamanın logo tasarımı.
- Entegrasyon problemlerinin çözülmesi.
- UML diyagramlarının çizimi.
- Kod hatalarının düzeltilmesi.
- Demo için main yazılması.
- Sunum ve belgeler için fikir alınması.
- Grup çalışması için gerekli teknolojilerin öğrenilmesi.

#### 3.3. Yapay Zekanın Katkısı

Yapay zekâ özellikle:

- Karmaşık algoritmaların daha hızlı anlaşılmasında
- Kod hatalarının daha kısa sürede çözülmesinde
- Alternatif mimari ve tasarım önerilerinin oluşturulmasında
- Konuların öğrenilmesinde

önemli katkılar sağlamıştır.

#### 4. Kullanıcı Kılavuzu

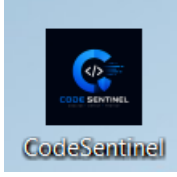
# CODE SENTINEL – Kullanım Kılavuzu

---

### Temel Kullanım Akışı

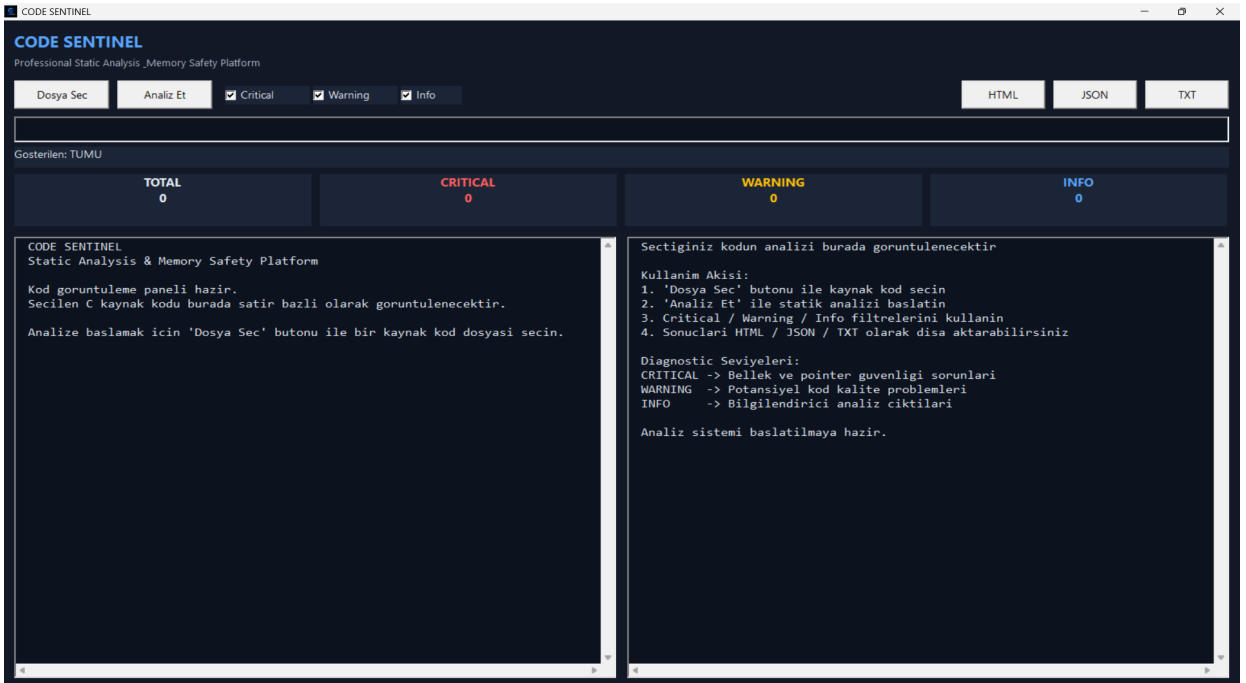
1. Programı çalıştırın
2. “Dosya Seç” butonuna basın
3. Kaynak kod dosyasını seçin
4. “Analiz Et” butonuna basın
5. Sonuçları inceleyin
6. İstenirse HTML / JSON / TXT olarak dışa aktarın

## 1.Giriş Yapma



CodeSentinel uygulamasını başlatmak için masaüstündeki uygulama simgesine çift tıklayın. Program açıldıktan sonra ana analiz ekranı görüntülenecektir.

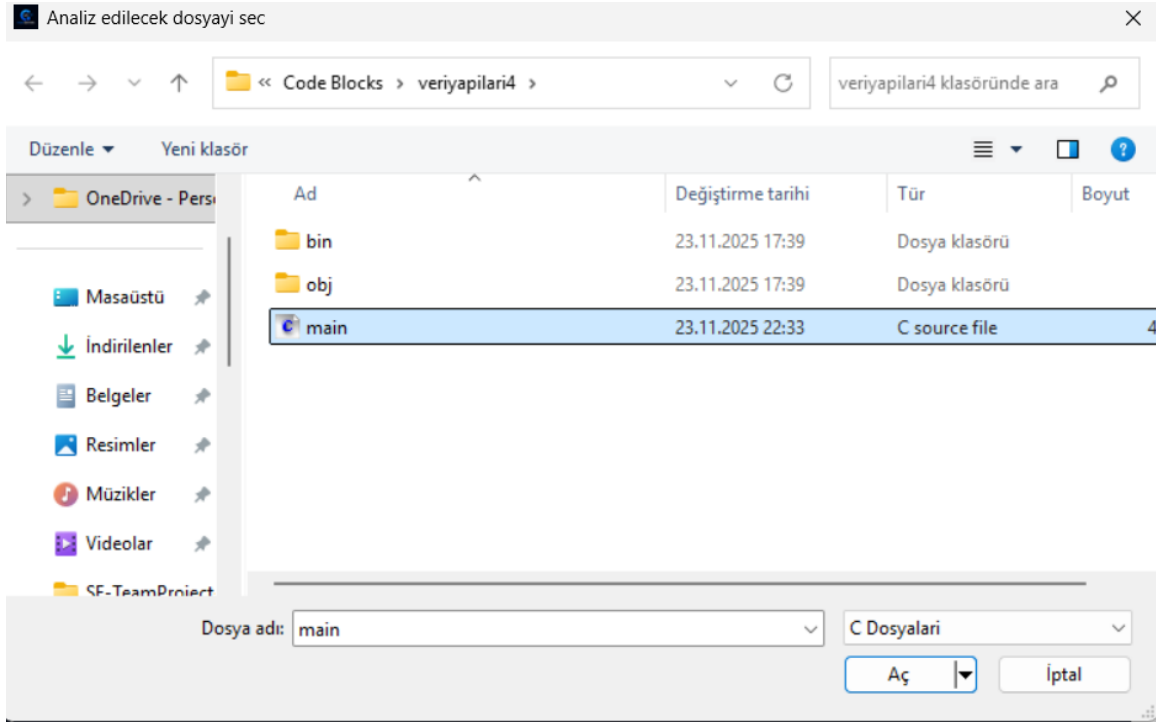
## 2. Ana Arayüz



Bu ekran uygulamanın ana arayüzünü göstermektedir.

| Bölüm                     | Açıklama                                   |
|---------------------------|--------------------------------------------|
| Dosya Seç                 | Analiz edilecek kaynak kod dosyasını seçer |
| Analiz Et                 | Statik analiz işlemini başlatır            |
| Critical / Warning / Info | Analiz filtreleme seçenekleri              |
| HTML / JSON / TXT         | Analiz raporunu dışa aktarma               |
| Sol Panel                 | Kaynak kod görüntüleme alanı               |
| Sağ Panel                 | Analiz çıktıları ve metrikler              |

### 3. Dosya Seçme İşlemi



Kullanıcı 'Dosya Seç' butonu ile analiz edilecek kaynak kod dosyasını seçer.

#### Adımlar

1. “Dosya Seç” butonuna tıklayın.
2. Açılan dosya penceresinden .c dosyasını seçin.
3. “Aç” butonuna basın.



## 4. Analiz Sonuçları

The screenshot displays the CODE SENTINEL Professional Static Analysis tool interface. The top bar shows the tool name and its purpose: "Professional Static Analysis ,Memory Safety Platform". Below this, there are tabs for "Dosya Sec", "Analiz Et", and a filter section with checkboxes for "Critical", "Warning", and "Info". The "Analiz Et" tab is active. The file path "C:\Users\ASUS\Desktop\Code Blocks\veriyapıları4\main.c" is entered in the file path field. The "Gosterilen: TUMU" dropdown is set to "TUMU".

The results are displayed in a table with four columns: TOTAL, CRITICAL, WARNING, and INFO. The values are: TOTAL 17, CRITICAL 0, WARNING 17, and INFO 0.

The left panel shows the source code of the program, which includes headers, a struct definition, and a function. The right panel shows the analysis results, including warnings and metrics.

**Warnings:**

- R003 | WARNING | Satir: 169 | Sutun: 14  
Mesaj: Magic number '6' tespit edildi. Sabit olarak tanımlayın.  
Kaynak: rule  
Kod: case 6:
- R004 | WARNING | Satir: 120 | Sutun: 1  
Mesaj: Fonksiyon 60 satir. Daha kucuk fonksiyonlara bolun.  
Kaynak: rule  
Kod: int main()
- R006 | WARNING | Satir: 131 | Sutun: 1  
Mesaj: Potansiyel sonsuz dongu tespit edildi.  
Kaynak: rule  
Kod: while(1){

**Metrics:**

===== METRIKLER =====  
Toplam Satir Sayisi: 179 satir -> Dosyadaki toplam satir sayisi  
Kod Satiri: 142 satir -> Bos olmayan ve yorum olmayan kod satiri sayisi  
Bos Satir: 36 satir -> Tamamen bos satir sayisi  
Yorum Satiri: 1 satir -> Yorum satiri sayisi  
Fonksiyon Sayisi: 10 adet -> Tespit edilen fonksiyon sayisi  
Struct Sayisi: 0 adet -> Tespit edilen struct sayisi  
Degisken Sayisi: 8 adet -> Struct alanlari haric degisken sayisi  
Ort. Fonksiyon Uzunlugu: 15.9 satir -> Fonksiyonların ortalama satir uzunlugu

Analiz tamamlandıktan sonra kaynak kod ve hata çıktıları ekranda görüntülenir.

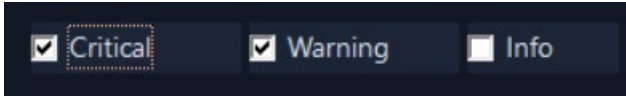
### Sol Panel

Kaynak kod satır bazlı olarak görüntülenir.

### Sağ Panel

Tespit edilen hata ve uyarılar görüntülenir.

## 5. Filtreleme Sistemi



Critical, Warning ve Info filtreleri kullanılarak analiz çıktıları filtrelenebilir.

Sistem tespit edilen problemleri üç kategoriye ayırır.

### Critical

Bellek ve pointer güvenliği ile ilgili kritik hatalar:

- Null Dereference
- Double Free
- Use After Free
- Dangling Pointer
- Memory Leak

### Warning

Kod kalite problemleri:

- Magic Number
- Uzun fonksiyonlar
- Sonsuz döngü ihtimali
- Kullanılmayan değişkenler

### Info

Bilgilendirici analiz çıktıları.

### Örnek Kullanımlar

- Sadece kritik hataları görmek için yalnızca **Critical** seçilir.
- Uyarıları gizlemek için **Warning** kaldırılır.
- Bilgilendirici mesajlar için **Info** aktif edilir.

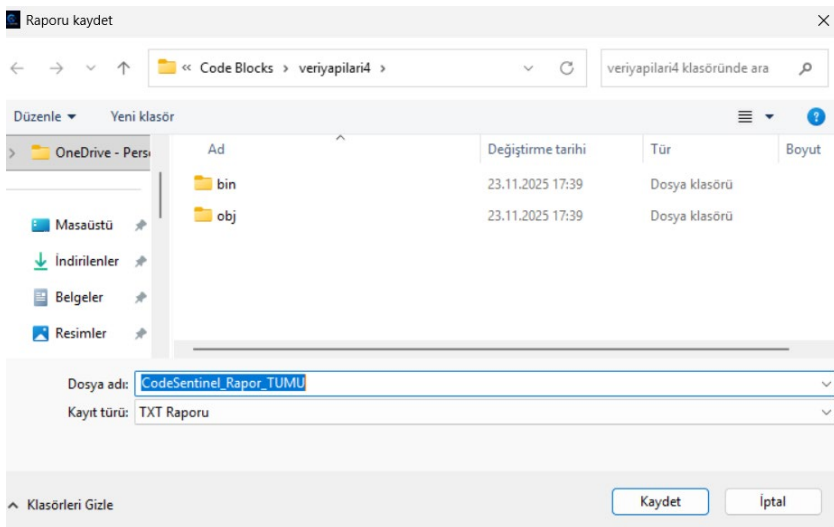
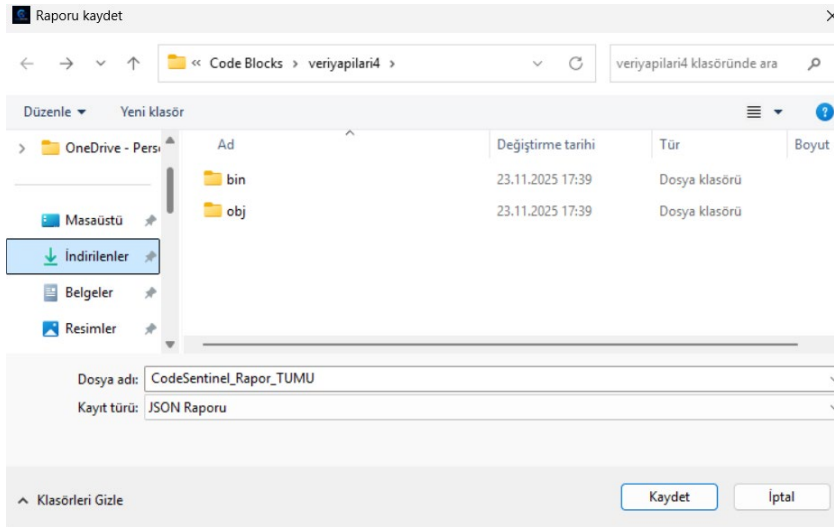
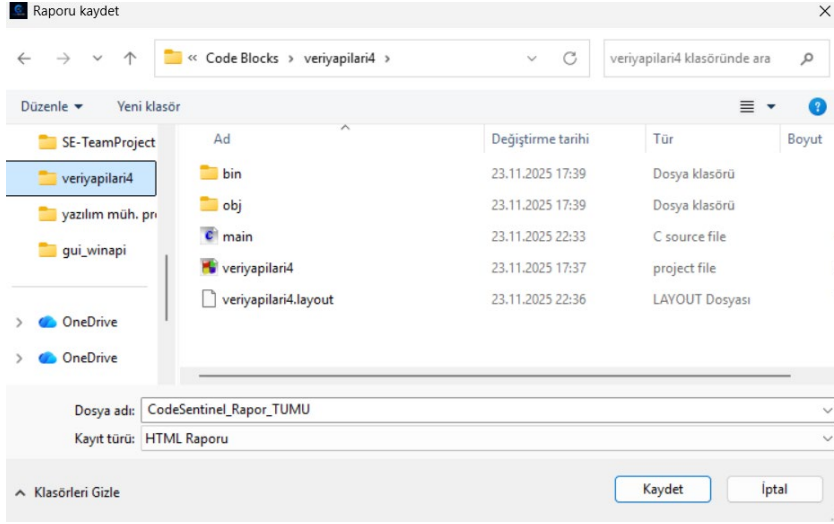
## 6.Rapor Dışa Aktarma

Sistem analiz sonuçlarını üç farklı formatta kaydedebilir:

| Format | Açıklama                                 |
|--------|------------------------------------------|
| HTML   | Web tarayıcısında görüntülenebilir rapor |
| JSON   | Veri işleme ve API kullanımı için        |
| TXT    | Düz metin raporu                         |

### Kullanım Adımları

- Program çalıştırılır.
- 'Dosya Seç' butonu ile kaynak kod dosyası seçilir.
- 'Analiz Et' butonu ile statik analiz başlatılır.
- Analiz sonuçları sağ panelde görüntülenir.
- İstenirse HTML, JSON veya TXT raporu oluşturulur.



İstenilen formatta rapor kayıt etme görselleri verilmiştir.

Bu kullanım kılavuzu, CODE SENTINEL statik analiz platformunun temel kullanım adımlarını açıklamak amacıyla hazırlanmıştır. Kılavuz içerisinde uygulamanın arayüz yapısı, dosya seçme işlemleri, analiz süreci, filtreleme sistemi ve rapor oluşturma adımları detaylı şekilde anlatılmıştır.

Kullanıcıların sistemi kolay ve doğru şekilde kullanabilmesi hedeflenerek tüm işlemler ekran görüntüleriyle desteklenmiş ve adım adım açıklanmıştır. Ayrıca analiz sonuçlarının yorumlanması, hata seviyelerinin anlaşılması ve rapor çıktılarının oluşturulması süreçleri de örneklerle birlikte gösterilmiştir.

Bu doküman sayesinde kullanıcıların CODE SENTINEL platformunu daha verimli kullanması, analiz süreçlerini daha iyi anlaması ve uygulamanın sunduğu özelliklerden maksimum düzeyde faydalanması amaçlanmaktadır.

## **5. Kurulum Talimatları**

Projeyi çalıştırmak isteyen bir kullanıcı linkten uygulamayı indirerek direkt kullanabilir.

## 6. Görev Dağılımı

| Üye                     | Analiz Raporu                          | Tasarım Raporu                                    | Implementasyon                      |
|-------------------------|----------------------------------------|---------------------------------------------------|-------------------------------------|
| Ferhat Enes KOCABIYIK   | Giriş , genel bakış ve önerilen sistem | Giriş , alt sistem ayrıştırması                   | Pointer Hataları ve Arayüz Tasarımı |
| Hacı Ömer OĞUZ          | Use Case diyagramları                  | Paketler/Katmanlar ve sınıf arayüzleri            | Syntax analyzer ve rule sınıfları   |
| Hüseyin Mert KANLIDERE  | Aktivite diyagramı                     | Donanım yazılım eşleşmesi ve kalıcı veri yönetimi | Infrastructure                      |
| Kemal BABAOĞLAN         | Sıra diyagramı                         | Erişim kontrolü ve güvenlik                       | Analyze Engine ve Reporting         |
| Mehmet Enes İYİŞENYÜREK | Sınıf diyagramı                        | Nesne tasarım kararları ve son nesne tasarımı     | Parser Geliştirme                   |
| Mehmet UÇAR             | Durum diyagramı                        | Tasarım desenleri                                 | Lexer Geliştirme                    |

## 7. Yapılanlar ve Yapılamayanlar

Sistem modüler ve katmanlı bir mimari ile tasarlanmıştır. Analiz sürecinin yönetimi merkezi bir AnalysisEngine sınıfı üzerinden gerçekleştirilmektedir. Kullanıcı arayüzü yalnızca AnalysisEngine ile iletişim kurmakta, diğer alt sistemler doğrudan erişilebilir olmamaktadır.

### Alt Sistemler

- **Application Core:** Sistemin yaşam döngüsünü yöneten ana katmandır.
- **File Management:** Dosya yükleme, uzantı kontrolü ve doğrulama işlemlerini gerçekleştirir.
- **Preprocessing & Lexer/Parser:** Yorum ayıklama, tokenization ve AST oluşturma işlemlerini yapar.
- **Analysis Engines:** Syntax analyzer, rule engine ve metric calculator modüllerini içerir.
- **Results & Reporting:** Analiz sonuçlarını yönetir ve rapor oluşturur.
- **User Interface:** Kullanıcı ile sistem arasındaki etkileşimi sağlar.

### Analiz Süreci

1. Kullanıcı .c uzantılı dosyayı yükler.
2. Dosya doğrulama işlemleri gerçekleştirilir.
3. Yorum satırları ayıklanır.
4. Lexer tarafından tokenization işlemi yapılır.
5. Parser tarafından AST oluşturulur.

6. Syntax analyzer syntax hatalarını tespit eder.
7. Rule engine kod kalitesi analizlerini gerçekleştirir.
8. Metric calculator metrikleri hesaplar.
9. Sonuçlar kullanıcı arayüzüne aktarılır.
10. Kullanıcı isterse rapor oluşturabilir.

### **Tamamlanan Özellikler**

- Dosya doğrulama sistemi geliştirildi
- Lexer ve tokenization altyapısı oluşturuldu
- Parser ve AST yapıları geliştirildi
- Syntax analyzer sistemi oluşturuldu
- Rule engine altyapısı geliştirildi
- Metrik hesaplama sistemi oluşturuldu
- TXT, JSON ve HTML raporlama desteği eklendi
- GitHub tabanlı ekip çalışma yapısı kuruldu

### **Tamamlanamayan Özellikler**

- İleri düzey güvenlik analizleri tamamlanamadı.
- Performans optimizasyonları sınırlı kaldı.
- Tam otomatik test sistemi geliştirilemedi.
- Hata navigasyon sistemi (Hataya tıklanınca hatanın olduğu satıra otomatik ulaşma)
- Hataları önceliklerine göre renklendirme.
- Kütüphanelerin kodun içine gömülmesi.

Bazı özelliklerin tamamlanamamasının temel nedenleri zaman yönetimi, modüller arası bağımlılıklar ve parser gibi karmaşık yapıların beklenenden daha uzun sürmesidir.

## **8. Sonuç ve Genel Değerlendirme**

Bu proje kapsamında C dili için çalışan temel seviyede bir statik kod analiz sistemi geliştirilmiştir. Sistem, syntax hatalarını ve bazı kod kalitesi problemlerini tespit ederek kullanıcıya rapor sunabilmektedir.

Proje sayesinde ekip üyeleri yazılım mühendisliği süreçleri, GitHub ekip çalışması, nesne yönelimli programlama, tasarım desenleri ve statik analiz mantığı konusunda önemli deneyimler kazanmıştır.

Gelecekte projeye daha gelişmiş parser desteği, daha fazla analiz kuralı, performans optimizasyonları, modern derleyicilere entegrasyonu ve gelişmiş kullanıcı arayüzü özellikleri eklenebilir.

## **9. GitHub Repository**

GitHub Repository Linki:

<https://github.com/mehmetenesiyisenyurek/SE-TeamProject-2026>

**SAYGILARIMIZLA  
CODESENTIEL EKİBİ**